Week 2: The Skeleton of the Grid

Graph Theory, Circuit Analysis, and Matrices

Instructor / Mentor

Kiki Research Training Program

Week 2

Contents

1. Graph Theory: Abstraction & Boxes
2. Circuit Analysis Intuition
3. From Physics to Matrices
4. Computational Efficiency
5. Vibe Coding Tasks
6. Homework

Graph Theory: Abstraction & Boxes

Saving Boxes: The Power of Abstraction

Last week, we learned about our limited "0/1 boxes" (Memory). How do we store a massive electrical grid efficiently?

- **Reality (Map):** A grid has GPS coordinates, wire curving around mountains, and specific geographic distances. Storing all this geometry requires millions of boxes.
- **Abstraction (Graph):** Graph Theory ignores the exact geometry. It only cares about **Nodes** (substations) and **Edges** (connections).

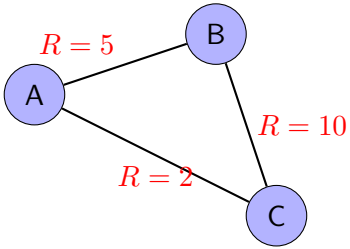
The Trade-off

By ignoring the shape of the wire and focusing only on *Topology* (who is connected to whom), we drastically reduce the number of boxes needed to model the system.

When Topology Isn't Enough: Adding Weights

Can we completely ignore distance? In a power grid, **no**.

- Longer wires mean higher **Resistance** and more energy lost as heat.
- To capture this physics without rebuilding the whole map, we add a single number to the edge: a **Weight**.
- A graph with weights perfectly balances mathematical simplicity with physical reality.



Circuit Analysis Intuition

Ohm's Law: The Engine of the Edge

What exactly happens on that "Edge" between Node A and Node B?

Ohm's Law: $V = I \times R$

- **Voltage (V):** The "pressure" pushing the electrons.
- **Current (I):** The actual flow/amount of electrons moving.
- **Resistance (R):** The friction in the wire (our Edge Weight).

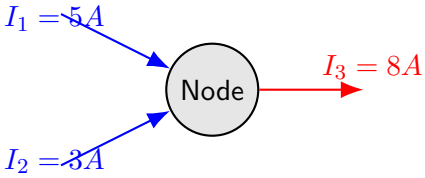
Analogy: Voltage is the water pump pressure. Current is the water flow. Resistance is how narrow the pipe is.

KCL: What Goes In, Must Come Out

Kirchhoff's Current Law (KCL): The sum of all currents entering a node must equal the sum of all currents leaving it. ($\sum I_{in} = \sum I_{out}$)

Why is this true?

- It's the *Conservation of Charge*.
- If more current entered a substation than left it, electrons would pile up infinitely. The node would become an unstable bomb of static charge!



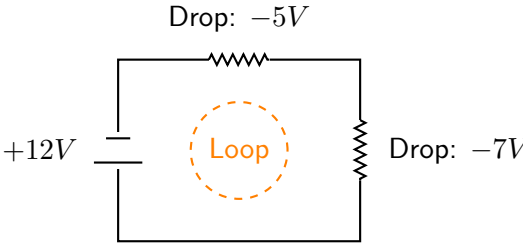
$$5A + 3A = 8A$$

KVL: The Mountain Hike

Kirchhoff's Voltage Law (KVL): The total voltage around any closed loop in a circuit must be zero. ($\sum V = 0$)

Why is this true?

- Voltage is a **Conservative Field** (like Gravity).
- **Hiking Analogy:** You walk up 500m, down 200m, and eventually return to your *exact starting point*. Your net elevation change must be exactly zero, otherwise you'd be floating!

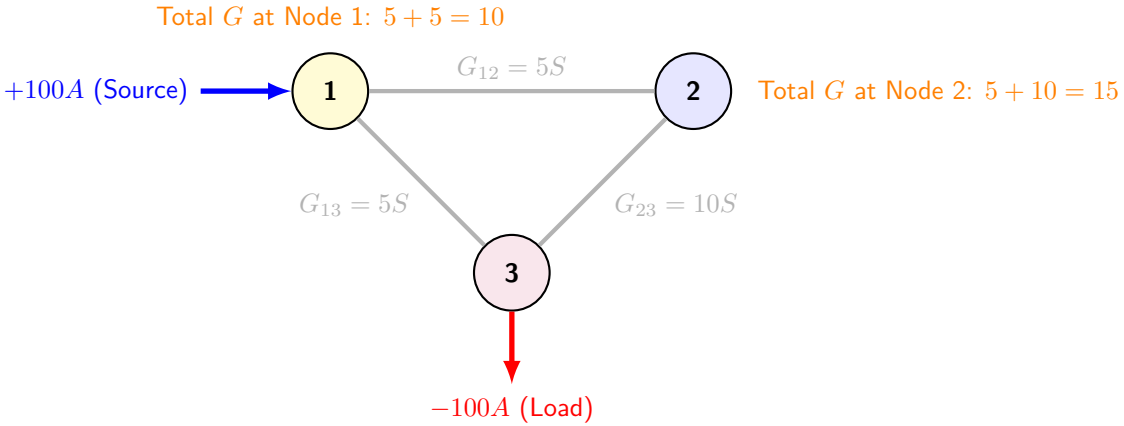


$$12 - 5 - 7 = 0$$

From Physics to Matrices

Visualizing the 3-Substation Grid

Before we look at the math, let's see the physical grid that created our equations. We use **Conductance ($G = 1/R$)** because it maps directly to our matrix numbers.



What is Conductance (G)?

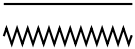
In grid research, we rarely talk about Resistance (R). Instead, we use its "inverse": **Conductance (G)**.

- **Intuition:** If Resistance is the "Obstruction," Conductance is the **"Smoothness"** of the path.
- **The Formula:** $G = \frac{1}{R}$
- **Why Matrices love G:**
  Standard Ohm's Law: $I = \frac{V}{R}$ (Division)
  Matrix-friendly version: $I = V \times G$ (Multiplication)
  Computers process multiplication significantly faster than division!

Fun Fact: The "Mho"

The modern unit for G is the **Siemens (S)**, but it used to be called the **Mho**.

- "Mho" is just **"Ohm"** spelled backwards!
- Even the symbol ($\mho$) is an upside-down Omega (Ω).



Resistance (Ω)

Conductance ($\mho$)

From the Grid to the Linear System

Using KCL ($\sum I = 0$) and Ohm's Law ($I = G \times \Delta V$), we write the flow for each node:

- **Node 1:** Own conductance ($10V_1$) minus shared flow with 2 and 3.
- **Node 2:** Own conductance ($15V_2$) minus shared flow with 1 and 3.

$$\begin{aligned} \mathbf{10}V_1 - \mathbf{5}V_2 - \mathbf{5}V_3 &= 100 \\ -\mathbf{5}V_1 + \mathbf{15}V_2 - \mathbf{10}V_3 &= 0 \\ -\mathbf{5}V_1 - \mathbf{10}V_2 + \mathbf{15}V_3 &= -100 \end{aligned}$$

The Logic

Diagonal: Sum of all wires connected to that node.
Off-Diagonal: The negative connection between two nodes.

Enter the Matrix: $Ax = b$

By separating the grid *structure* from the *variables*, we write the whole system as one clean object:

$$\begin{bmatrix} 10 & -5 & -5 \\ -5 & 15 & -10 \\ -5 & -10 & 15 \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ V_3 \end{bmatrix} = \begin{bmatrix} 100 \\ 0 \\ -100 \end{bmatrix}$$

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$$

- $\mathbf{A}$: The Adjacency Matrix (Grid Topology)
- $\mathbf{x}$: The unknown Voltages
- $\mathbf{b}$: The known power inputs/outputs

Solving the System: The Matrix Inverse

In standard 9th-grade algebra, if $5x = 10$, you divide to get $x = 10/5$.
In matrix algebra, we don't "divide". We multiply by the **Inverse Matrix** (A^{-1}).

The Matrix Solution

If we have the equation: $\mathbf{A}x = \mathbf{b}$
Multiply both sides by A^{-1}: $\mathbf{A}^{-1}\mathbf{A}x = \mathbf{A}^{-1}\mathbf{b}$
Since $A^{-1}A$ cancels out: $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$

Intuition: The inverse matrix A^{-1} mathematically "undoes" the grid's complex topology, instantly giving us the voltage at every single node simultaneously!

What is a Matrix Inverse (A^{-1})?

In regular math, the inverse of 5 is $1/5$ because $5 \times \frac{1}{5} = 1$. In matrix world, we have the **Identity Matrix (I)** which acts like the number "1".

The Definition

If A is a square matrix, its inverse A^{-1} is the unique matrix such that:

$$A \cdot A^{-1} = I$$

Note: Only square matrices (same rows and columns) can have an inverse, and even then, only if they aren't "broken" (singular).

The Identity Matrix (I)

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1s on the diagonal, 0s everywhere else.

How do we calculate A^{-1}?

For a 9th grader, calculating a 3×3 inverse by hand is a marathon, but the "recipe" involves two main ingredients:

1. **The Determinant ($\det(A)$):** A single number that summarizes the matrix.
   - If $\det(A) = 0$, the inverse **does not exist**. The grid is physically disconnected!
2. **The Adjugate Matrix:** A "transformed" version of A based on its internal cross-multiplications.

The Formula

$$A^{-1} = \frac{1}{\det(A)} \times \text{Adjugate}(A)$$

*In research, we usually let computers use **Gaussian Elimination** (row reduction) because it is much more efficient than this formula!*

Computational Efficiency

Why Matrices? Symbolic vs. Numeric

Why do we force computers to use matrices instead of human algebra (like substitution and elimination)?

Symbolic Calculation (Human Way)

- The computer stores "V_1" as a text string and moves it around in memory.
- Requires thousands of logic rules to track moving variables.
- Terribly slow for large systems.

Matrix Calculation (Computer Way)

- Drops all text symbols. Only processes pure grid numbers (Floats).
- The memory is packed perfectly in RAM (boxes side-by-side).
- The CPU handles rows and columns in bulk using hardware acceleration.

Quantifying the Efficiency

Imagine solving a 1,000-node town grid.

- **The Human Way:** Parsing text equations for 1,000 variables would take massive software overhead.
- **The Matrix Way (O(N^3)):** Solving an $N \times N$ matrix takes roughly N^3 operations. For 1,000 nodes, that's ≈ 1 billion floating-point operations (FLOPs).

The Result

A modern CPU does *hundreds of billions* of operations per second. What would take a human thousands of lifetimes takes Python/MATLAB **less than 0.05 seconds**. This is why matrices are king!

Vibe Coding Tasks

Vibe Coding Task 1: Building the Graph Topology

Goal: Use AI to generate code that creates a power grid with resistance weights and draws it on screen.

Prompting Hints for your AI

1. Tell the AI: "Write a Python script using the `networkx` library."
2. Instruct it: "Create a 5-node graph. Add edges between the nodes, and give each edge a 'weight' attribute representing electrical resistance (e.g., 5 Ohms, 10 Ohms)."
3. Ask the AI: "Use `matplotlib` to draw the graph and visibly display the weights on the lines."

Observe: Look at how the AI handles the `add_edge` function. You don't need to memorize the syntax, just verify the logic!

Vibe Coding Task 2: Solving the Matrix

Goal: Use AI to build matrix A and vector b, and solve for x without using human algebra.

Prompting Hints for your AI

1. Tell the AI: "Write a Python script using `numpy`."
2. Give it the exact numbers: "Create a 3x3 array A: [[10,-5,-5], [-5,15,-10],[-5,-10,15]]. Create a 1D array b: [100, 0, -100]."
3. Instruct it: "Use the `numpy.linalg.solve` function to solve for x (the voltages). Print the result."

Why `linalg.solve`? Ask your AI why we use `solve()` instead of literally calculating the matrix inverse (`np.linalg.inv`). The answer will teach you a lot about numerical stability!

Homework

Homework: Manual Coding Training

From Edge List to Adjacency Matrix

You are given an "Edge List" representing a power grid:

```
edges = [[0, 1],[1, 2], [0, 2], [2, 3]]
```

Task: You must generate a 4×4 Adjacency Matrix containing 0s and 1s based on that list.

Rules: STRICTLY NO AI. No NetworkX. No NumPy auto-conversions. You must write the nested `for`-loops by reading official Python documentation!

Thank you!

Next week: Academic Literature Searches & Database Architectures.